

**Методичні вказівки до виконання лабораторних робіт
з дисципліни: Поглиблене вивчення JavaScript
Лабораторна робота №4
Розробка застосунку реєстрації на події (розширені можливості Vue)**

1.1 Мета роботи

Закріплення та поглиблення знань з розробки застосунків на Vue 3, засвоєння принципів використання Composition API, організації логіки за допомогою composables, реалізація багатосторінкових застосунків, а також формування навичок роботи з асинхронними операціями та збереженням даних.

1.2 Загальні відомості

На сучасному етапі розвитку вебтехнологій застосунки характеризуються складною структурою, наявністю декількох сторінок, інтерактивністю та необхідністю збереження стану між сесіями. Реалізація таких систем потребує використання сучасних підходів до організації коду та управління даними.

Фреймворк Vue 3 надає широкі можливості для створення подібних застосунків. Зокрема, використання Composition API дозволяє структурувати код за функціональними блоками, а застосування маршрутизації забезпечує організацію навігації між різними частинами застосунку.

У межах даної лабораторної роботи створюється застосунок для перегляду подій та реєстрації користувачів на них, що наближений до реальних умов розробки.

1.3 Теоретичні відомості

Composition API є сучасним підходом до організації логіки у Vue-застосунках. Його використання дозволяє групувати пов'язані частини функціоналу разом, що значно підвищує читабельність та спрощує підтримку коду.

Важливим елементом цього підходу є composables — функції, які інкапсулюють певну логіку (наприклад, роботу з даними або API) та

можуть бути використані у різних компонентах. Це забезпечує повторне використання коду та зменшує його дублювання.

Для створення багатосторінкових застосунків використовується маршрутизація, яка дозволяє організувати навігацію між різними представленнями (views) без перезавантаження сторінки.

У сучасних застосунках важливим є збереження даних користувача. Для цього може використовуватися локальне сховище браузера (localStorage), що дозволяє зберігати інформацію між сеансами.

Особливу увагу слід приділити асинхронним операціям. Взаємодія із сервером (або її імітація) виконується із використанням асинхронних функцій. Для покращення користувацького досвіду застосовується підхід оптимістичного інтерфейсу, при якому результат дії відображається одразу, ще до завершення операції.

Додаткові можливості Vue, такі як Teleport та KeepAlive, дозволяють оптимізувати структуру інтерфейсу та підвищити ефективність роботи застосунку.

1.4 Інструкція зі встановлення та налаштування середовища

Для виконання лабораторної роботи необхідно створити новий проєкт Vue 3 із використанням інструмента Vite або використати попередній проєкт.

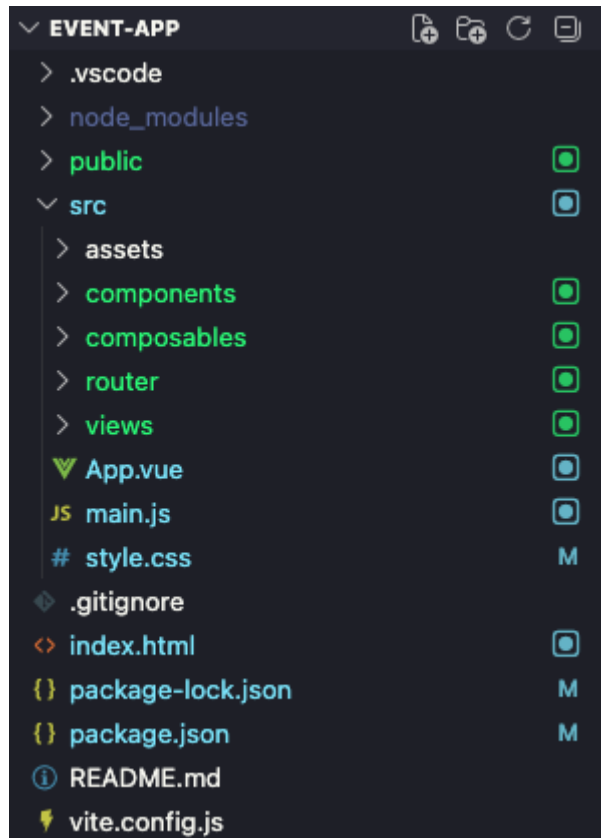
Після створення проєкту необхідно встановити залежності та додатково підключити бібліотеку маршрутизації:

```
npm install vue-router
```

Після цього необхідно налаштувати маршрутизацію, створивши відповідний конфігураційний файл та підключивши роутер до застосунку.

1.5 Структура проєкту

Для реалізації поставленого завдання рекомендується використовувати модульну структуру проєкту:



components/ — компоненти інтерфейсу;
views/ — окремі сторінки застосунку;
router/ — налаштування маршрутизації;
composables/ — функції для повторного використання логіки;
assets/ — статичні ресурси.

Такий підхід забезпечує розділення відповідальностей та покращує масштабованість застосунку.

1.6 Порядок виконання роботи

Виконання лабораторної роботи здійснюється у такій послідовності:

1. Створити проєкт Vue 3 та встановити необхідні залежності.
2. Підключити та налаштувати Vue Router для організації навігації.
3. Створити основні сторінки застосунку:
 - сторінку списку подій;
 - сторінку детальної інформації про подію;
 - сторінку реєстрації.
4. Реалізувати відображення списку подій.

5. Організувати перехід до сторінки конкретної події.
6. Реалізувати відображення детальної інформації про подію.
7. Створити форму реєстрації користувача на подію.
8. Реалізувати збереження введених даних у реактивних змінних.
9. Реалізувати валідацію форми.
10. Створити composables для:
 - роботи зі списком подій;
 - обробки реєстрації;
 - збереження даних у localStorage.
11. Реалізувати збереження даних у localStorage.
12. Реалізувати асинхронну логіку реєстрації (імітацію запиту до сервера).
13. Реалізувати оптимістичне оновлення інтерфейсу.
14. Реалізувати обробку помилок та відкат змін у разі невдачі.
15. Реалізувати відображення списку зареєстрованих користувачів.
16. За потреби використати додаткові можливості Vue (Teleport, KeepAlive).
17. Перевірити працездатність застосунку.

1.7 Приклад реалізації застосунку

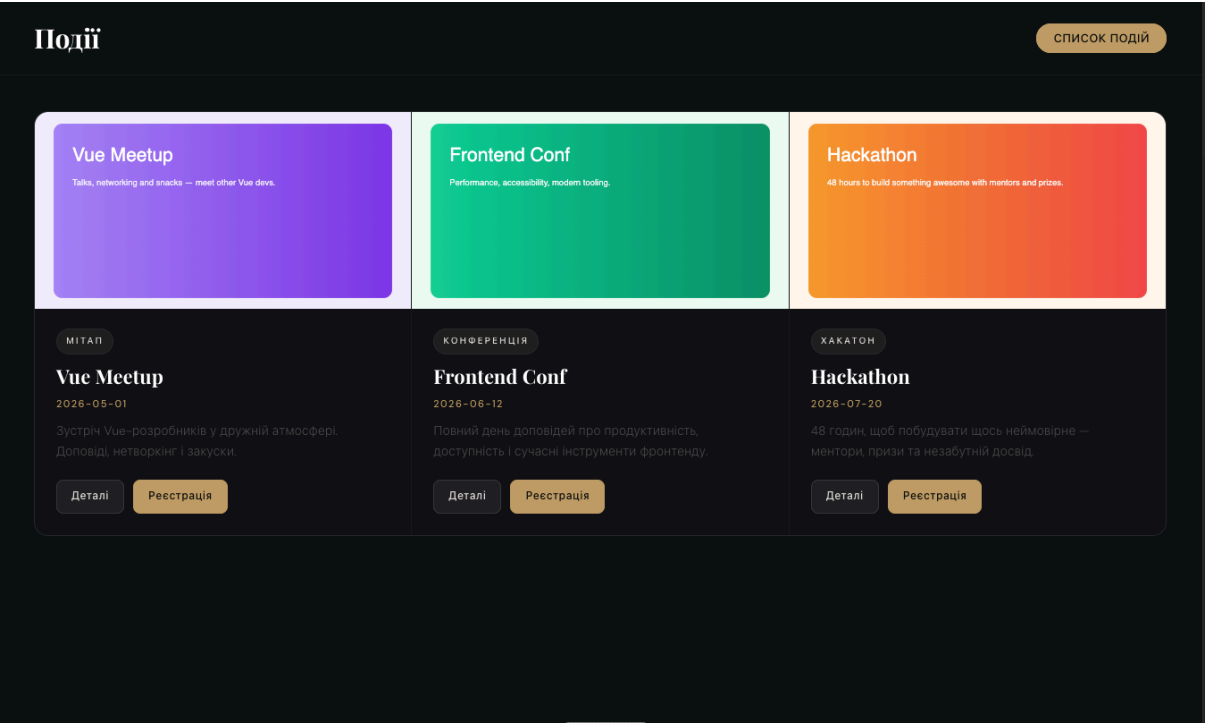
Реалізація застосунку базується на використанні Composition API та розподілі логіки між компонентами та composables.

Список подій зберігається у відповідному composable та використовується у різних частинах застосунку. Дані про реєстрації користувачів зберігаються у localStorage, що дозволяє зберігати стан між перезавантаженнями сторінки.

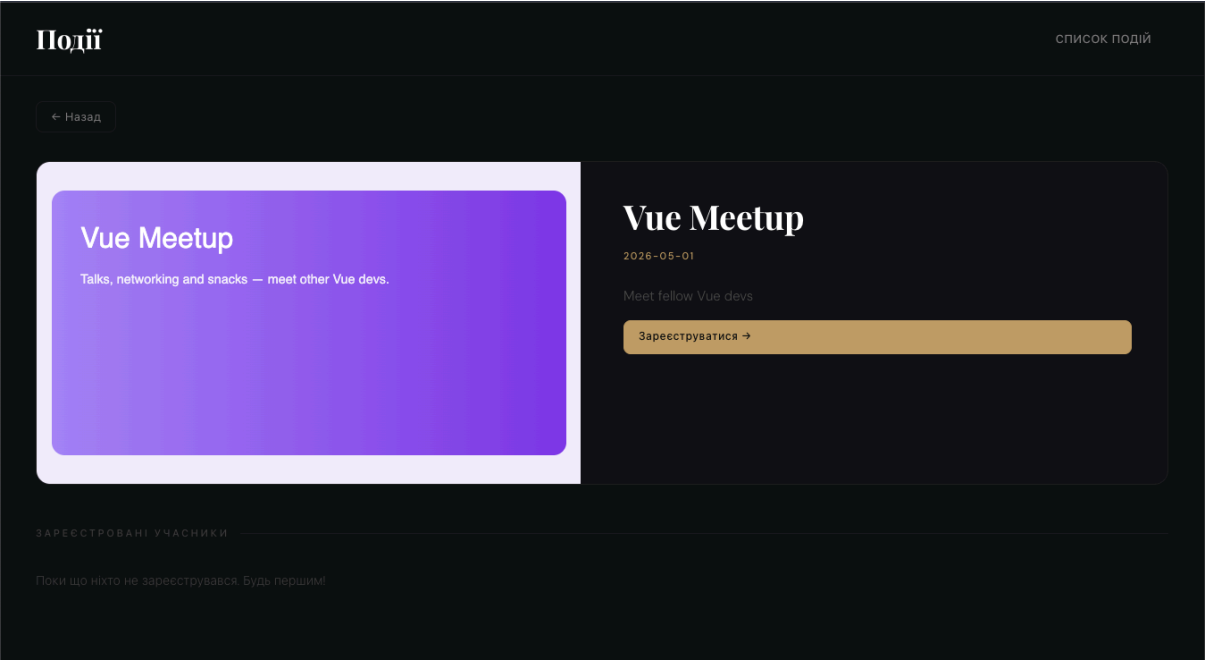
Форма реєстрації використовує реактивні змінні для збереження введених даних та виконує їх перевірку перед відправкою. Логіка відправки винесена у composable, що забезпечує її повторне використання.

Оптимістичний підхід дозволяє відобразити результат дії користувача до завершення асинхронної операції. У разі виникнення помилки виконується відкат змін та відображається повідомлення.

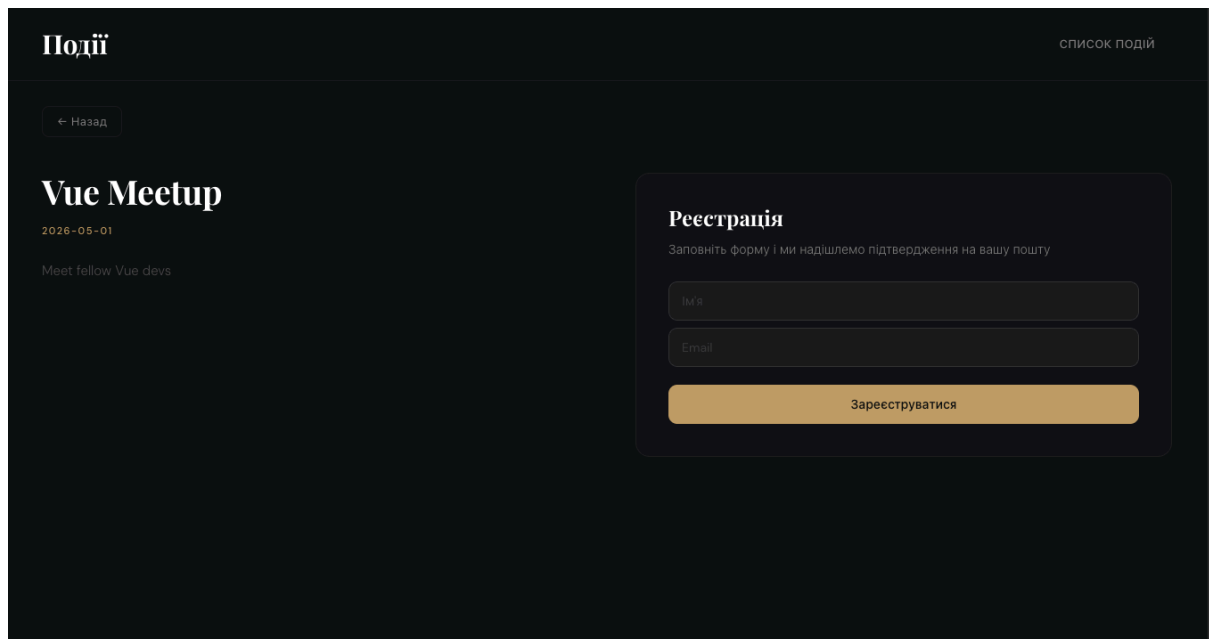
Головна сторінка:



Деталі:



Сторінка реєстрації:



1.8 Завдання до виконання

У процесі виконання лабораторної роботи необхідно:

- реалізувати багатосторінковий застосунок;
- створити список подій;
- реалізувати перегляд деталей події;
- реалізувати реєстрацію на подію;
- реалізувати збереження даних;
- використати Composition API;
- використати composables;
- реалізувати асинхронну логіку;
- реалізувати обробку помилок;
- реалізувати оптимістичний інтерфейс.

1.9 Вимоги до захисту

Усі лабораторні роботи виконуються у вигляді окремих проєктів або модулів одного проєкту. Код має бути структурованим, із чітким поділом на компоненти та дотриманням принципів чистого коду.

Результати роботи студент повинен представити у вигляді:

- вихідного коду

- короткого звіту з описом мети, теоретичних відомостей та реалізації
- демонстрації працездатності застосунку

Під час захисту особлива увага приділяється розумінню механізмів асинхронного програмування, логіці організації запитів та обробки відповідей, а також умінню пояснити власний код.

1.10 Оцінювання лабораторних робіт

Кожна лабораторна робота оцінюється максимально у **25 балів**. Оцінка залежить від повноти виконання завдання, працездатності застосунку та здатності студента пояснити свою реалізацію під час захисту.

Чим повніше реалізовано функціонал і стабільніше працює додаток, тим вищий бал. Частково виконані або некоректні роботи оцінюються нижче.

1.11 Контрольні питання

1. Що таке Composition API?
2. Що таке composables?
3. Призначення Vue Router.
4. Як реалізується робота з localStorage?
5. Що таке оптимістичний інтерфейс?
6. Як обробляються помилки в асинхронних операціях?